Jordan Pauzie, Julian Savini, Charlie Zhang
CS: 151-2 Special Topics in Data Infrastructure and Systems
Professor Lillethun & Professor Sundar
11/24/25

Project 4 Beta Writeup

Executive Overview:

In our design document, we initially outlined three areas of improvement including function inlining for frequently-called functions such as get_piece_at(), caching board values and improving branch flow in move_gen.c. Some of these optimizations were successfully implemented, and along the way, the idea of replacing the insertion sort algorithm with merge sort algorithm also came up.

Performance Optimizations

With that in mind, we inlined get_piece_at() and square_to_index() to great success, as we were able to obtain an absolute speedup of 62%. These results make sense as well given our preliminary profiling report; even at low depths, functions such as get_piece_at() and square_to_index() are called numerous times per search. By inlining these functions, we removed many call boundaries and enhanced the compiler's ability to aggressively optimize our code.

During our initial sprint of optimizations, we made the decision to implement several optimizations (including the inlining described above) concurrently. In doing so, we were met with persistent illegal move errors upon running the autotester. This resulted in our group having to backtrack to an untouched baseline version of our code, and methodically work our way back up, testing each change one at a time. With this, we discovered that the "optimizations" streamlining branch flow in generate_all() in move_gen.c were at fault. There had been an error in boolean logic conversion when we tried refactoring the if-statement logic into something more compiler friendly which resulted in the list of possible moves including illegal ones. With that "optimization" removed, we were left with three viable optimizations: function inlining, monarch position caching, and replacing insertion sort with merge sort.

One of the optimizations we implemented was monarch caching. With a board of length *n*, O(n^2) lookups were called to identify the monarch positions. By storing the monarch positions in the result, and then updating the position in set_piece_at(), the get_monarch() function lookups run in O(1) time. Additionally, set_piece_at() simply uses conditional statements to identify the type of piece, in order to update the result. Keep in mind, we did add fallback just in case a position is invalid. As we were observing illegal moves after combining multiple optimizations, we added this function just in case. Our profiling doesn't indicate the get_monarch() function is consuming much compute, so we assume fallback lookups are not occurring.

The sorting optimization involved replacing the inefficient O(N^2) insertion sort previously used in searchPV() with a more scalable O(NlogN) merge sort. Because search Root() relies on searchPv() to evaluate and order moves , this improvement directly accelerates the root-level

move ordering process as well.In addition, the Merge Sort implementation is structured so that it can be parallelized in the future, enabling further speedups on multicore systems.

Identification of Bottlenecks:
With the conclusion of our beta code, we decided to perform a profiling report for each individual ./leiserchess command akin to the one we performed for the design document. The commands we tested were display, eval, generate, status, go, move and perft. After investigating the profiling data for each command, we discovered that the following functions monopolized the CPU:

- The get_piece_at() function was, before optimization, 21.2% of compute for running perft. Now, after optimization, get_piece_at() only is 6.18% of compute for running perft. Inlining was very effective.
- Originally, square_to_index() was taking 7.5% of compute for running perft. Now, after inlining, square_to_index() is only taking 1.78% of compute for running perft.

As we continue to reduce large bottlenecks, there are still bottlenecks [much smaller] that still arise. Here are a couple of functions which are current bottlenecks. In the “Ideals for Final Submission” section, we outline how we can perhaps improve these functions.

- generate_all() [12.73% cpu for perft]
- recompute_zob() [10.21% cpu for perft]
- toggle_piece() [9.75% cpu for perft]

Post Beta Retrospective:
Our team was eager to start making an impact in this project. That being said, we learned a lot about project management along the way. We tested a lot of optimizations in bulk at the start of the project, and since these were minor optimizations, we did not expect anything to break. Unfortunately, the monarch was generating illegal moves, and this caused us to have to rebranch from main, and A/B test our optimizations. This was an exhaustive process, and slowed us down in beta. Regardless, it was good to learn from our mistakes, get proper test metrics, and thus we had proof and an understanding of how our optimizations worked.

Ideas for the Final Submission:

- Merge sort is an easily parallelizable sorting algorithm, so implementing this functionality using OpenCilk will be one of our first optimizations.
- In generate_all(), we believe that there are several lines of code (mostly logical operators such as if-else statements) which can be streamlined to improve compiler branching and vectorization. Furthermore, some struct field calls can be cached and reused within loops to improve memory locality.
- To eliminate board scanning in generate_all(), we can store the pieces, and their respective locations. We can also try loop unrolling.
- Static inline toggle_piece()
- Batch operations when checking multiple adjacent squares
- Static inline move_of()

Performance Progress:

Original bot vs. optimized bot :

Version	Absolute Speedup	Changes (if any)
Baseline	0%	
Version 1	+34%	Inlining get_piece_at()
Version 2	+62%	Inlining square_to_index()
Version 3	+74%	Implemented monarch caching
Version 3	+74%	Reducing extraneous square_to_index call
Version 5	+77%	Replaced insertion sort with merge sort

Team Dynamics/Division of Work:
Remaining consistent with our design document, our team did a great job prioritizing the project throughout the week, and the three of us were pretty consistently on call together, testing new ideas and debugging. Generally, we were open to an idea proposed in the project 4 guideline, saying each section should have a “CEO” throughout the project. Generally speaking, Charlie had ownership over the mergesort algorithm. Julian had ownership over the monarch position caching, and scripting to test depth %. Jordan had ownership over testing and attacking performance bottlenecks with inlining. But, we all spent time helping each other debug and test these features. Overall, we are happy with our beta submission, as we were able to comfortably do a congruent depth search in less than half the time.

Project Log:

#	Date	Duration	Description
1	Fri, Nov. 14	1	Formed team and created team contract
2	Tue, Nov. 18	3	Started the design document and began completing checkoff
3	Wed, Nov. 19	2	Finished design document and checkoff
4	Sat, Nov. 22	2	Implemented optimizations such as function inlining, monarch caching, merge sort
5	Sat, Nov. 22	2	Debugging illegal move issue which stemmed from one of our optimizations
6	Sun, Nov. 23	3	Solved the illegal move bug by removing a piece of improperly optimized code
7	Mon, Sept 24	2	Added script to test suite; see percentage of improvement
8	Mon, Sept 24	2	Completed code for beta and significantly improved absolute

			performance in terms of speed
9	Mon, Sept 24	2	Beta Writeup

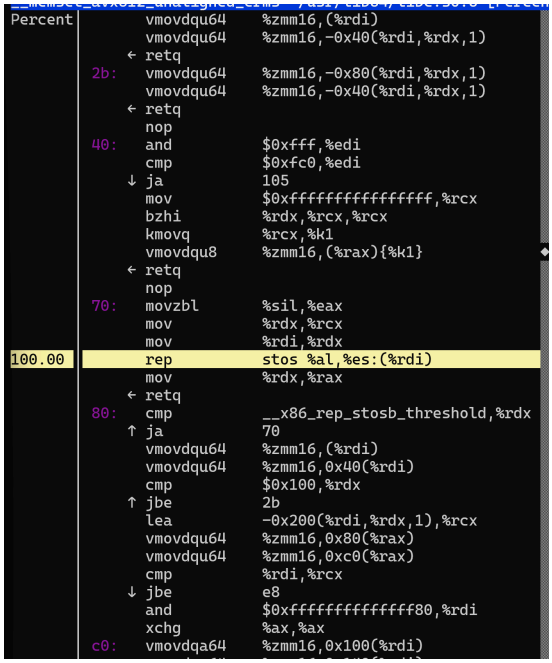
Total Hours: 19

Additional Information & Acknowledgements:
No additional information needed.

Appendix 1 (Profiling Data):
PERFT:

```
Samples: 62K of event 'cycles:Pu', Event count (approx.): 466453088444
Overhead  Command      Shared Object         Symbol
  12.73%  leiserchess  leiserchess           [.] generate_all
  10.21%  leiserchess  leiserchess           [.] recompute_zob
  10.09%  leiserchess  leiserchess           [.] square_of
   9.75%  leiserchess  leiserchess           [.] toggle_piece
   7.67%  leiserchess  leiserchess           [.] get_piece_at
   6.80%  leiserchess  leiserchess           [.] piece_index
   6.71%  leiserchess  leiserchess           [.] check_square
   6.18%  leiserchess  leiserchess           [.] get_piece_at
   5.70%  leiserchess  leiserchess           [.] make_sortable
   4.10%  leiserchess  leiserchess           [.] move_of
   2.86%  leiserchess  leiserchess           [.] player_wins
   2.13%  leiserchess  leiserchess           [.] take_cardinal_step
   2.10%  leiserchess  leiserchess           [.] get_piece_at
   1.78%  leiserchess  leiserchess           [.] square_to_index
   1.54%  leiserchess  leiserchess           [.] take_ordinal_step
   1.31%  leiserchess  leiserchess           [.] get_file_offset
   1.12%  leiserchess  leiserchess           [.] ordinal_dir_to_index
   1.02%  leiserchess  libc.so.6             [.] __memset_avx512_unaligned_erms
   0.97%  leiserchess  leiserchess           [.] get_rank_offset
   0.88%  leiserchess  libc.so.6             [.] __memmove_avx512_unaligned_erms
   0.58%  leiserchess  leiserchess           [.] make_ordinal
   0.53%  leiserchess  leiserchess           [.] square_eq
   0.34%  leiserchess  leiserchess           [.] set_piece_at
   0.30%  leiserchess  leiserchess           [.] move_piece
   0.29%  leiserchess  leiserchess           [.] step_laser
   0.24%  leiserchess  leiserchess           [.] fire_laser
   0.23%  leiserchess  leiserchess           [.] cardinal_of_file_dir
   0.20%  leiserchess  leiserchess           [.] cardinal_of_rank_dir
   0.16%  leiserchess  leiserchess           [.] cardinal_dir_to_index
   0.15%  leiserchess  leiserchess           [.] execute_turn
   0.14%  leiserchess  leiserchess           [.] perft_search
   0.14%  leiserchess  leiserchess           [.] get_laser_origin
   0.14%  leiserchess  leiserchess           [.] get_monarch
   0.11%  leiserchess  leiserchess           [.] check_board_zob
   0.08%  leiserchess  leiserchess           [.] move_eq
   0.08%  leiserchess  leiserchess           [.] toggle_color_to_move
   0.07%  leiserchess  leiserchess           [.] get_move
   0.07%  leiserchess  leiserchess           [.] check_valid_square
   0.06%  leiserchess  leiserchess           [.] is_game_over
   0.06%  leiserchess  leiserchess           [.] color_to_move_of
   0.06%  leiserchess  leiserchess           [.] rotate_ordinal
   0.05%  leiserchess  leiserchess           [.] swap_color_to_move
   0.04%  leiserchess  leiserchess           [.] rotate_file_right
   0.04%  leiserchess  leiserchess           [.] rotate_piece
   0.03%  leiserchess  leiserchess           [.] opp_color
   0.03%  leiserchess  leiserchess           [.] rotate_rank_right
   0.03%  leiserchess  leiserchess           [.] rotate_cardinal
   0.02%  leiserchess  [unknown]             [k] 0xffffffff8b601122
   0.02%  leiserchess  leiserchess           [.] fil_of
   0.02%  leiserchess  leiserchess           [.] rnk_of
   0.01%  leiserchess  leiserchess           [.] memcpy@plt
   0.01%  leiserchess  leiserchess           [.] cardinal_dir_eq
   0.00%  leiserchess  leiserchess           [.] update_victim
   0.00%  leiserchess  libc.so.6             [.] __vfprintf_internal
   0.00%  leiserchess  libxalt_init.so       [.] __XALT_host_switch_to_buffer
   0.00%  leiserchess  ld-linux-x86-64.so.2  [.] intel_check_word.constprop.0
   0.00%  leiserchess  ld-linux-x86-64.so.2  [.] __GI___tunables_init
   0.00%  leiserchess  ld-linux-x86-64.so.2  [.] _dl_start
Tip: Create an archive with symtabs to analyse on other machine: perf archive
```

ANNOTATING MEMSET:



DISPLAY:

```
Samples: 2K of event 'cycles:Pu', Event count (approx.): 488343856
Overhead  Command      Shared Object          Symbol
  97.43%  leiserchess  libc.so.6             [.] __memset_avx512_unaligned_erms
   2.21%  leiserchess  [unknown]             [k] 0xffffffff8b601122
   0.11%  leiserchess  ld-linux-x86-64.so.2  [.] _dl_check_map_versions
   0.10%  leiserchess  leiserchess           [.] init_zob
   0.04%  leiserchess  leiserchess           [.] init_board
   0.04%  leiserchess  leiserchess           [.] strcmp@plt
   0.04%  leiserchess  ld-linux-x86-64.so.2  [.] init_cpu_features.constprop.0
   0.01%  leiserchess  ld-linux-x86-64.so.2  [.] _dl_sysdep_parse_arguments
   0.00%  leiserchess  libc.so.6             [.] _IO_getline_info
   0.00%  leiserchess  ld-linux-x86-64.so.2  [.] _dl_start
   0.00%  leiserchess  libc.so.6             [.] _IO_default_uflow
   0.00%  leiserchess  ld-linux-x86-64.so.2  [.] _start
   0.00%  leiserchess  libc.so.6             [.] _IO_file_underflow@@GLIBC_2.2.5
   0.00%  leiserchess  libc.so.6             [.] read
```

GO:

```
Samples: 2K of event 'cycles:Pu', Event count (approx.): 527741834
Overhead  Command      Shared Object          Symbol
  92.64%  leiserchess  libc.so.6             [.] __memset_avx512_unaligned_erms
   1.92%  leiserchess  [unknown]             [k] 0xffffffff8b601122
   1.45%  leiserchess  leiserchess           [.] toggle_piece
   0.92%  leiserchess  leiserchess           [.] recompute_zob
   0.70%  leiserchess  leiserchess           [.] get_piece_at
   0.54%  leiserchess  leiserchess           [.] check_square
   0.43%  leiserchess  leiserchess           [.] piece_index
   0.38%  leiserchess  leiserchess           [.] square_of
   0.25%  leiserchess  leiserchess           [.] square_to_index
   0.14%  leiserchess  libc.so.6             [.] __memmove_avx512_unaligned_erm
   0.14%  leiserchess  leiserchess           [.] move_piece
   0.12%  leiserchess  leiserchess           [.] mcede_pair
   0.11%  leiserchess  ld-linux-x86-64.so.2  [.] _dl_lookup_direct
   0.07%  leiserchess  ld-linux-x86-64.so.2  [.] intel_check_word.constprop.0
   0.06%  leiserchess  leiserchess           [.] ordinal_dir_to_index
   0.05%  leiserchess  libc.so.6             [.] munmap_chunk
   0.05%  leiserchess  leiserchess           [.] generate_all
   0.02%  leiserchess  ld-linux-x86-64.so.2  [.] __GI___tunables_init
   0.00%  leiserchess  libc.so.6             [.] _IO_getline_info
   0.00%  leiserchess  ld-linux-x86-64.so.2  [.] _dl_start
   0.00%  leiserchess  libc.so.6             [.] _IO_default_uflow
   0.00%  leiserchess  libc.so.6             [.] _IO_file_underflow@@GLIBC_2.2.
   0.00%  leiserchess  ld-linux-x86-64.so.2  [.] _start
   0.00%  leiserchess  libc.so.6             [.] read
```

EVAL:

```
Samples: 2K of event 'cycles:Pu', Event count (approx.): 495096505
Overhead  Command      Shared Object         Symbol
  96.99%  leiserchess  libc.so.6             [.] __memset_avx512_unaligned_erms
   2.42%  leiserchess  [unknown]             [k] 0xffffffff8b601122
   0.26%  leiserchess  leiserchess           [.] check_square
   0.14%  leiserchess  libxalt_init.so       [.] __XALT_xmalloc_xalt_1_5
   0.09%  leiserchess  ld-linux-x86-64.so.2  [.] handle_intel.constprop.0
   0.07%  leiserchess  ld-linux-x86-64.so.2  [.] do_lookup_x
   0.02%  leiserchess  ld-linux-x86-64.so.2  [.] __GI___tunables_init
   0.01%  leiserchess  ld-linux-x86-64.so.2  [.] _dl_start
   0.00%  leiserchess  libc.so.6             [.] _IO_getline_info
   0.00%  leiserchess  libc.so.6             [.] _IO_default_uflow
   0.00%  leiserchess  ld-linux-x86-64.so.2  [.] _start
   0.00%  leiserchess  libc.so.6             [.] _IO_file_underflow@@GLIBC_2.2.
   0.00%  leiserchess  libc.so.6             [.] read
```

GENERATE:

```
Samples: 2K of event 'cycles:Pu', Event count (approx.): 488201134
Overhead  Command      Shared Object         Symbol
  97.76%  leiserchess  libc.so.6             [.] __memset_avx512_unaligned_erms
   1.94%  leiserchess  [unknown]             [k] 0xffffffff8b601122
   0.15%  leiserchess  leiserchess           [.] myrand
   0.09%  leiserchess  ld-linux-x86-64.so.2  [.] intel_check_word.constprop.0
   0.04%  leiserchess  ld-linux-x86-64.so.2  [.] check_match
   0.02%  leiserchess  ld-linux-x86-64.so.2  [.] __GI___tunables_init
   0.00%  leiserchess  ld-linux-x86-64.so.2  [.] _dl_start
   0.00%  leiserchess  ld-linux-x86-64.so.2  [.] _start
```

MOVE A3B4:

```
Samples: 2K of event 'cycles:Pu', Event count (approx.): 488549010
Overhead  Command      Shared Object         Symbol
  97.22%  leiserchess  libc.so.6             [.] __memset_avx512_unaligned_erms
   2.40%  leiserchess  [unknown]             [k] 0xffffffff8b601122
   0.11%  leiserchess  ld-linux-x86-64.so.2  [.] _dl_relocate_object
   0.11%  leiserchess  leiserchess           [.] myrand
   0.06%  leiserchess  ld-linux-x86-64.so.2  [.] update_active.constprop.0
   0.04%  leiserchess  libc.so.6             [.] buffered_vfprintf
   0.04%  leiserchess  leiserchess           [.] toggle_piece
   0.01%  leiserchess  ld-linux-x86-64.so.2  [.] _dl_sysdep_parse_arguments
   0.00%  leiserchess  ld-linux-x86-64.so.2  [.] _dl_start
   0.00%  leiserchess  ld-linux-x86-64.so.2  [.] _start
```

STATUS:

```
Samples: 2K of event 'cycles:Pu', Event count (approx.): 503503946
Overhead  Command      Shared Object         Symbol
  97.65%  leiserchess  libc.so.6             [.] __memset_avx512_unaligned_erms
   1.99%  leiserchess  [unknown]             [k] 0xffffffff8b601122
   0.13%  leiserchess  libc.so.6             [.] _int_malloc
   0.10%  leiserchess  libc.so.6             [.] getenv
   0.08%  leiserchess  ld-linux-x86-64.so.2  [.] intel_check_word.constprop.0
   0.02%  leiserchess  libc.so.6             [.] _IO_default_uflow
   0.02%  leiserchess  ld-linux-x86-64.so.2  [.] __GI___tunables_init
   0.00%  leiserchess  libc.so.6             [.] _IO_getline_info
   0.00%  leiserchess  ld-linux-x86-64.so.2  [.] _dl_start
   0.00%  leiserchess  libc.so.6             [.] _IO_file_underflow@@GLIBC_2.2.
   0.00%  leiserchess  ld-linux-x86-64.so.2  [.] _start
   0.00%  leiserchess  libc.so.6             [.] read
```